CSC317 Computer Graphics

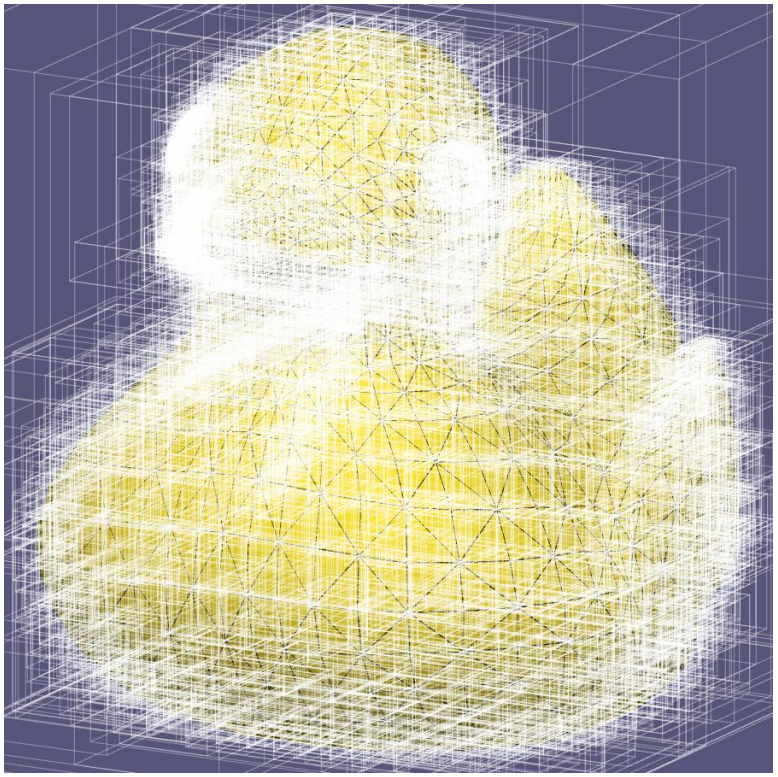
Tutorial: Assignment 3

Bounding Volume Hierarchy (AABB Tree)

Assignment 3: Bounding Volume Hierarchy

Build an **AABB tree** over triangles or points, then use it to speed up three queries:

- Ray $\leftrightarrow$ mesh intersection
- Nearest point in a point cloud
- Overlapping triangle pairs between two meshes



Why do we need a BVH?

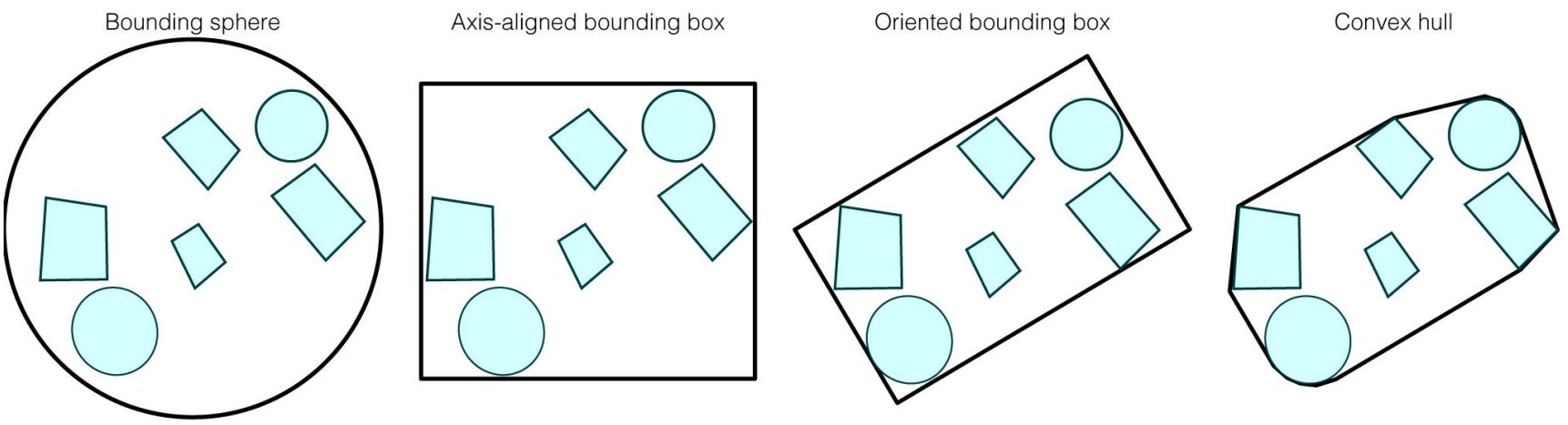
- Brute force: every ray is tested against **every** triangle → O(n) per query
- Most triangles are nowhere near the ray
- Idea: throw away a whole group of triangles with **one** cheap test

8.31 s	0.02 s	0.08 s
brute force	build tree	use tree

```
python main.py --mode rays   (1000 rays, 668 triangles)
```

Idea 1: Bounding volumes

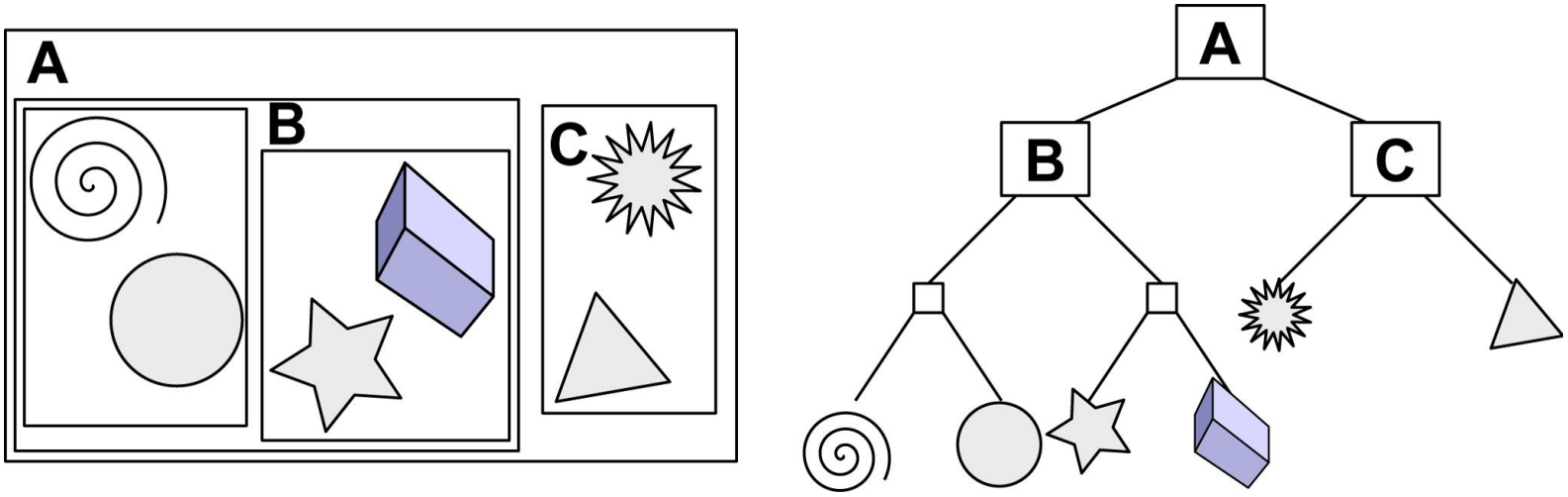
- Wrap the geometry in a simple shape → a **cheap** test that answers:
  - "definitely miss" → skip everything inside, no exact tests
  - "maybe" → now do the exact tests
- Tighter shape → fewer false "maybe"s, but a slower test
- **AABB** is the sweet spot: two corners `min_corner`, `max_corner`, per-axis arithmetic



← looser fit, cheaper test

tighter fit, costlier test →

Idea 2: Put boxes inside boxes



- One box around the whole mesh only helps if the ray misses **everything**
  - Hit the box → still n exact tests
- So nest boxes: each level can rule out a whole group at once
  - Like binary search: ~log n box tests instead of n triangle tests
- **Object partitioning**: each object sits in exactly one leaf
  - Sibling boxes may **overlap** — but no object ever gets cut

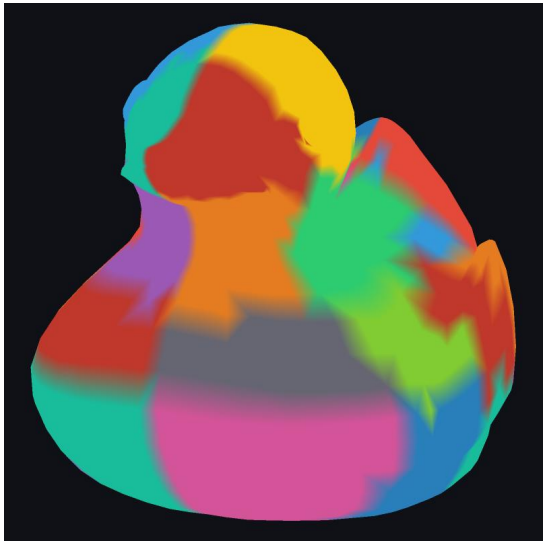
How the tree is built



depth 1



depth 3



depth 5

- Box around all objects → cut at the **midpoint of the longest axis**
- Each object goes left / right by its **box center**
- Recurse until a node holds 1 or 2 objects

Using the tree: three queries

query	traversal	pruning rule
Ray ↔ mesh	depth-first recursion	skip boxes the ray misses
Nearest point	priority queue (heapq)	visit the closest box first
Mesh ↔ mesh	queue over pairs of nodes	skip pairs of boxes that don't overlap

Building costs O(n log n); each query then behaves like O(log n) instead of O(n).

Setup

From the repository root, once:

```
./init.sh                          # macOS / Linux
.\init.ps1                         # Windows PowerShell
```

Every new terminal:

```
source venv/bin/activate           # macOS / Linux
.\venv\Scripts\Activate.ps1        # Windows
```

Windows refuses to run the script?

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

- You only edit `src/` (one function per file). Don't touch `backend/`, `tests/`.

Checking your work

```
python run_tests.py --only ray_intersect_box    # one function
python run_tests.py                             # all 12
python check_my_work.py                         # as marks
python main.py --mode rays --headless           # demos:
python main.py --mode distances --headless      #  tree vs brute
python main.py --mode pairs --headless          #  force + timing
```

- Test data is readable in `tests/expected/` (`.txt` / `.png`)
- Grade: **30%** public tests + **70%** same checks on hidden inputs
  - Passing the public tests ≠ no bugs — edge cases matter
- Submit all `src/*.py` on MarkUs

A good order to implement

Part 1 · rays

1. `ray_intersect_triangle`
2. `ray_intersect_triangle_mesh_brute_force` ← needs 1
3. `insert_box_into_box, insert_triangle_into_box`
4. `AABBTree` ← needs 3
5. `ray_intersect_box`
6. `AABBTree_ray_intersect` ← needs 1, 4, 5

Part 2 · distances

7. `nearest_neighbor_brute_force`
8. `point_box_squared_distance`
9. `point_AABBTree_squared_distance` ← needs 4, 8

Part 3 · pairs

10. `box_box_intersect`
11. `find_all_intersecting_pairs_using_AABBTrees` ← needs 4, 10

1 · Ray–triangle intersection

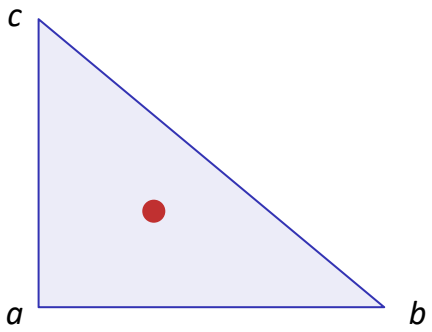
`src/ray_intersect_triangle.py` · 10 pts

test: python run_tests.py --only ray_intersect_triangle

Ray point = point on the triangle's plane:

$$e + t\,d = a + \beta\,(b - a) + \gamma\,(c - a)$$

- 3 equations (x, y, z), 3 unknowns (t, β, γ) → 3×3 linear system
- Solve with Cramer's rule (FCG §4.4.2)
- Hit only if:
  - `min_t <= t < max_t`
  - `0 <= γ <= 1` and `0 <= β <= 1 - γ` (inside the triangle)



1 · Ray–triangle: structure

```
def ray_intersect_triangle(ray, A, B, C, min_t, max_t):
    # 1. columns of the 3x3 matrix: from A, B, C and ray.direction
    # 2. right-hand side: from A and ray.origin
    # 3. det = determinant of the matrix
    #    det == 0  ->  ray parallel / flat triangle  ->  ???
    # 4. Cramer: t, gamma, beta = (sub-determinant) / det
    # 5. reject t outside [min_t, max_t)
    # 6. reject gamma outside [0, 1]
    # 7. reject beta  outside [0, 1 - gamma]
    return hit, t
```

2 · Ray vs whole mesh (brute force)

`src/ray_intersect_triangle_mesh_brute_force.py` · 10 pts

test: python run_tests.py --only ray_intersect_triangle_mesh_brute_force

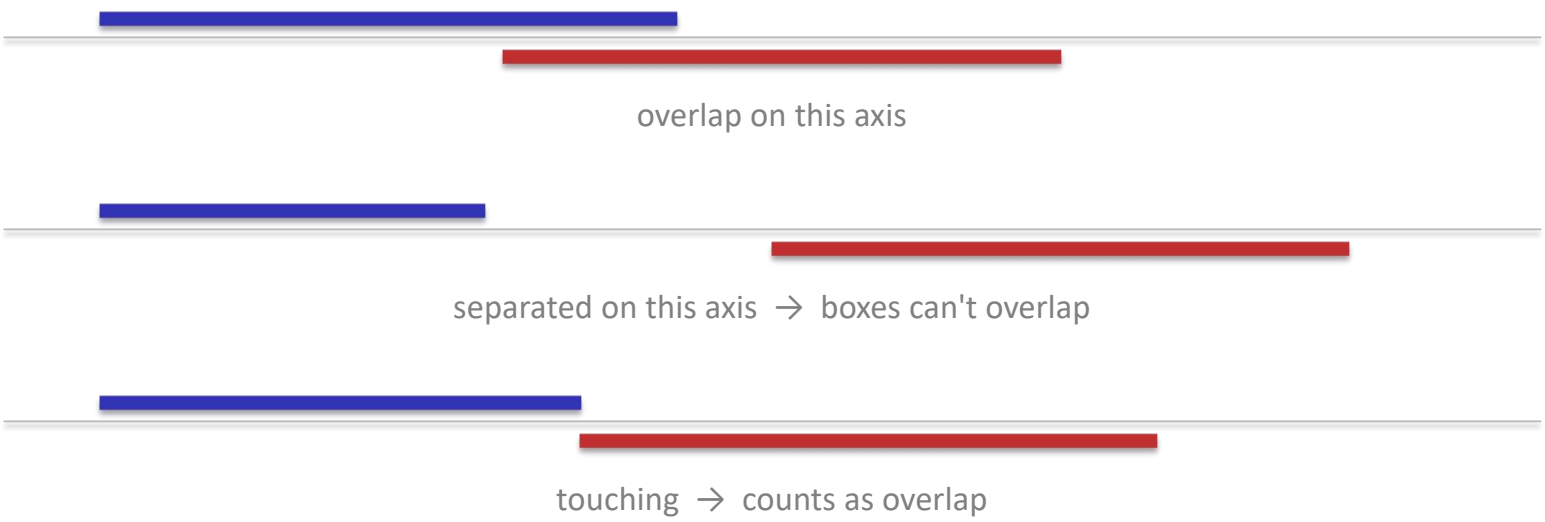
```
def ray_intersect_triangle_mesh_brute_force(ray, V, F, min_t, max_t):
    best_t, best_f = inf, -1
    for f in range(len(F)):
        a, b, c = ???                          # the 3 corners of face f
        hit, t = ray_intersect_triangle(ray, a, b, c, min_t, ???)
        if hit and ???:                        # closer than best so far?
            best_t, best_f = t, f
    return ???, best_t, best_f
```

- `V`: (n, 3) vertex positions, `F`: (m, 3) vertex indices per face
- This is the **reference answer** for the tree later — get it right now
- Return face index `-1` when nothing is hit

10 · Box–box overlap

`src/box_box_intersect.py` · 2 pts

test: python run_tests.py --only box_box_intersect

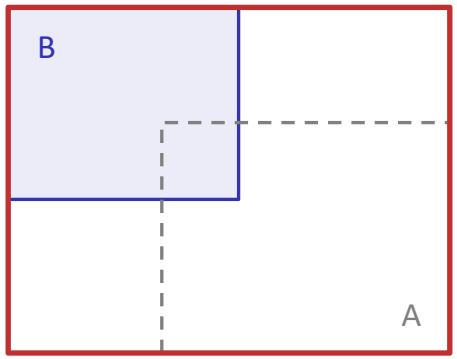



```
for each axis c:
    if one box ends before the other begins on c:  ???
return ???
```

3 · Growing a box

`src/insert_box_into_box.py`, `src/insert_triangle_into_box.py` · 10 + 10 pts

- Empty box: min `+inf`, max `-inf`
- Growing an empty box by anything → tightest box around it
- Mutate the given box **in place**, and also return it



B after

```
def insert_box_into_box(A_box, B_box):
    B_box.min_corner = ???    # per component
    B_box.max_corner = ???
    return B_box

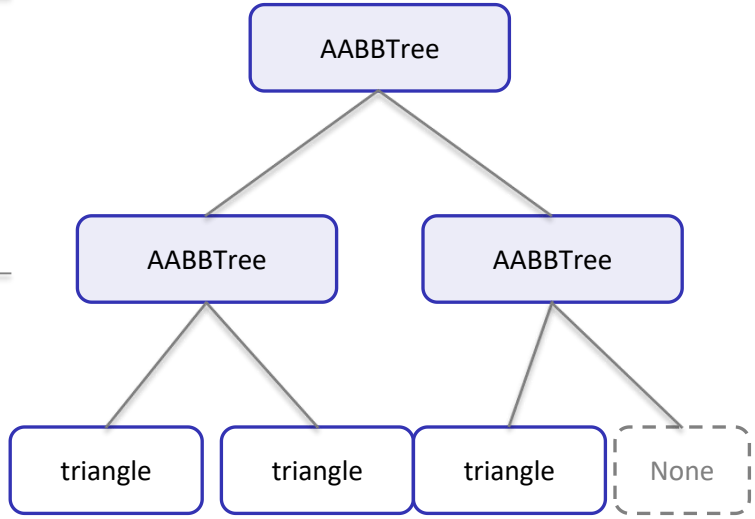
def insert_triangle_into_box(a, b, c, box):
    # same idea, with the 3 corners a, b, c
```

4 · AABBTree: what a node stores

`src/AABBTree.py` · 15 pts

test: python run_tests.py --only AABBTree

```
class AABBTree(Object):
    box          # around everything below
    left         # AABBTree or leaf
    right        # AABBTree, leaf, or None
    depth        # root = 0
    num_leaves   # objects below this node
```



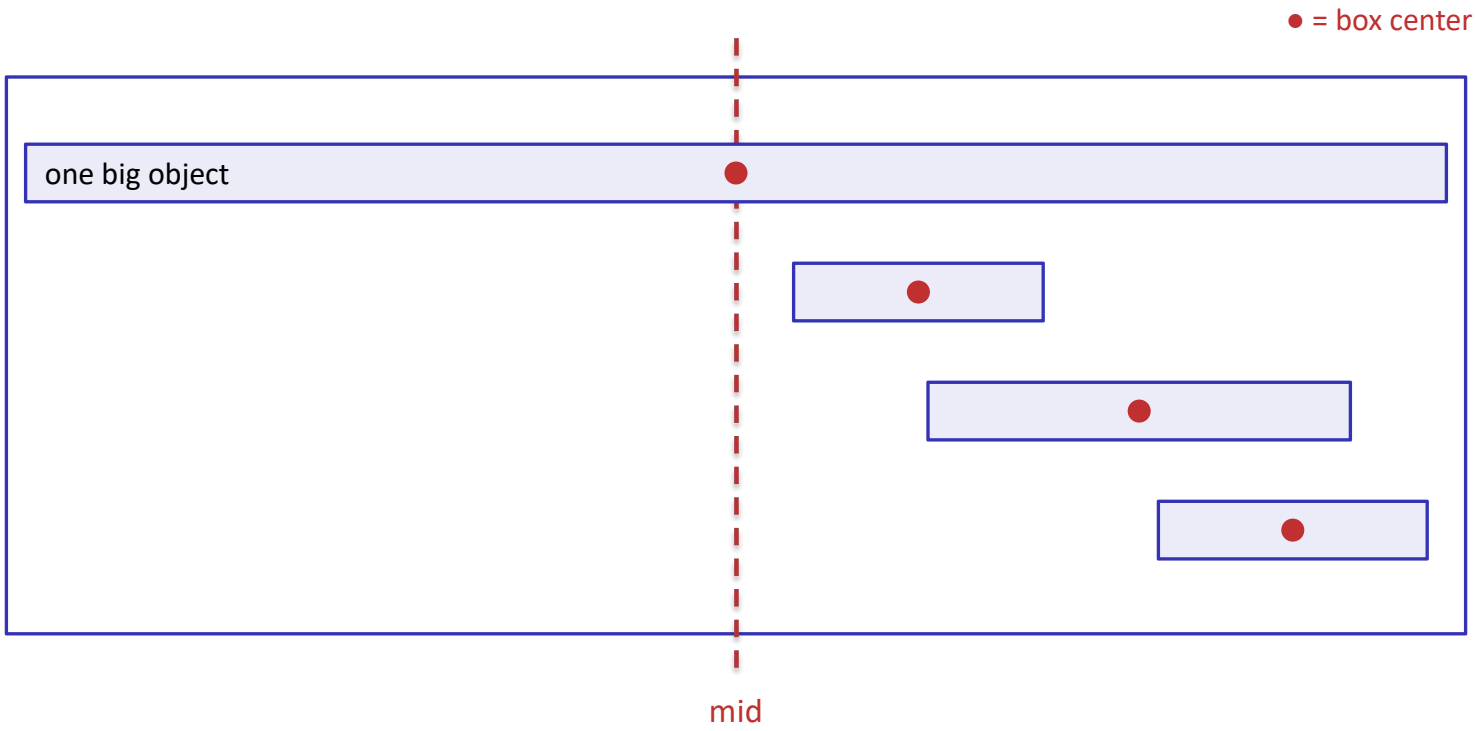
- `AABBTree` **is an** `Object` → a child can be a subtree or a leaf
- Leaves are the input objects themselves (`MeshTriangle`, `CloudPoint`)
- Every later query only reads these 5 fields

4 · AABBTree: building

```
def __init__(self, objects, depth=0):
    self.box <- empty box grown by every obj.box    # insert_box_into_box
    if len(objects) <= 2:
        left, right <- the objects themselves        # stop here
        return
    axis <- longest side of self.box
    mid  <- middle of self.box along axis
    L, R <- split objects: obj.box.center()[axis] < mid ?
    if L or R is empty: ???                          # next slide
    self.left  = AABBTree(L, depth + 1)
    self.right = AABBTree(R, depth + 1)
    self.num_leaves <- ???
```

- Set **all five** attributes on every node, including the small ones
- `np.argmax` of the box extent gives the longest axis

4 · AABBTree: the empty-side trap

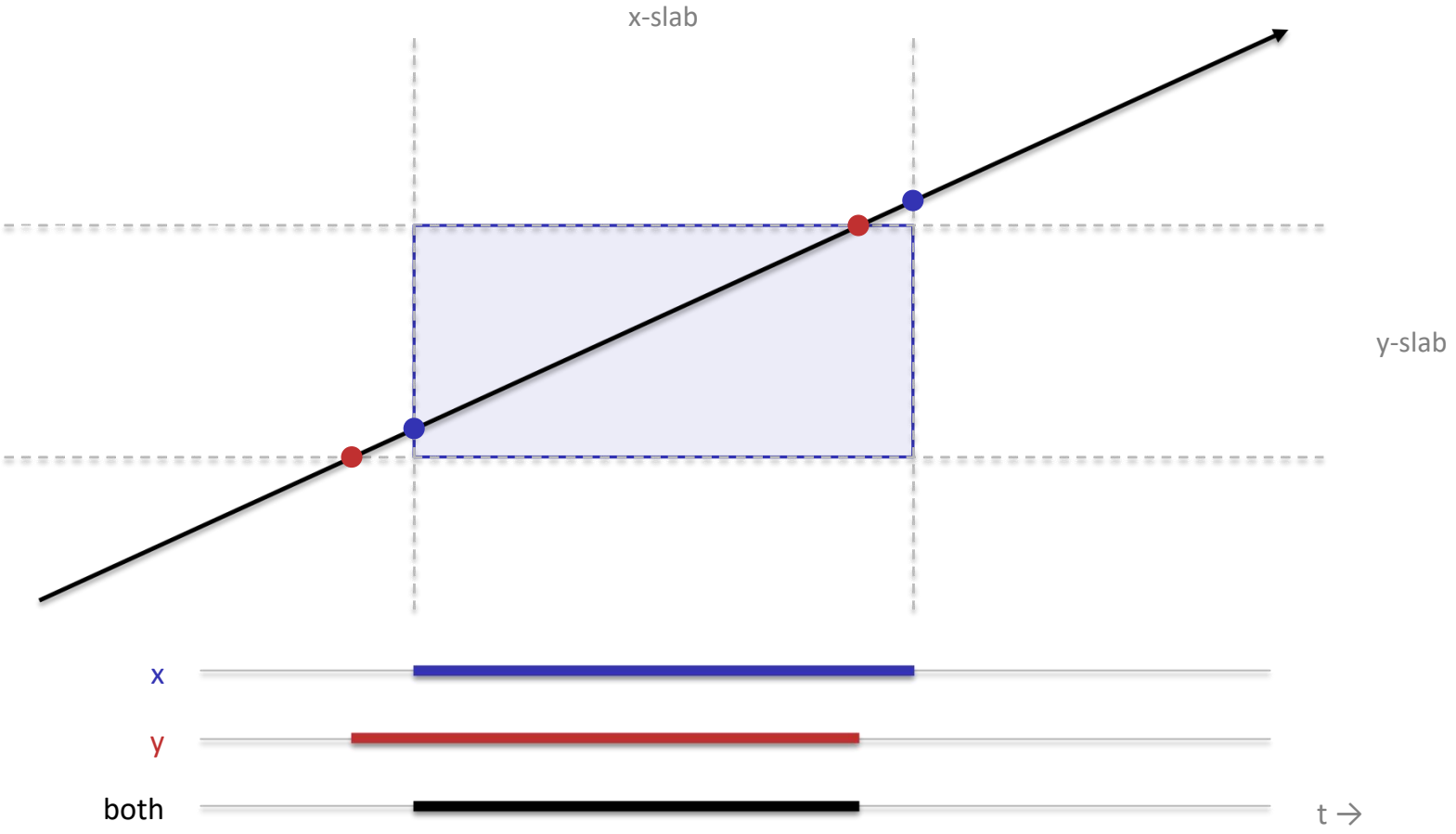


- Every center is ≥ mid → **left side is empty** → same call again, forever
- Also happens with many identical points
- Fix: move one object across so both sides are non-empty

5 · Ray–box: the slab method

`src/ray_intersect_box.py` · 10 pts

test: python run_tests.py --only ray_intersect_box



5 · Ray–box: structure

```
def ray_intersect_box(ray, box, min_t, max_t):
    lo, hi = min_t, max_t                # running interval
    for c in range(3):
        t0 <- t where the ray reaches box.min_corner[c]
        t1 <- t where the ray reaches box.max_corner[c]
        if ???: swap t0, t1              # which plane comes first?
        lo, hi <- intersect [lo, hi] with [t0, t1]
        if the interval is empty: ???
    return ???
```

- Negative `direction[c]` flips which plane is entered first
- `direction[c] == 0`: think about what `1 / 0` does
- The box is **solid**: a ray starting inside is a hit

6 · Ray vs tree

`src/AABBTree_ray_intersect.py` · 10 pts

test: python run_tests.py --only AABBTree_ray_intersect

```
def AABBTree_ray_intersect(root, ray, min_t, max_t):
    if ray misses root.box:                 # ray_intersect_box
        return False, ???, None             # whole subtree skipped
    best <- no hit yet
    for child in (root.left, root.right):
        if child is None: continue
        if child is an AABBTree:  recurse into it
        else:                      ray_intersect_triangle(...)
        keep the closer hit; ??? max_t for the next child
    return hit, t, leaf_object_that_was_hit
```

- Return the **leaf** (`MeshTriangle`) — caller reads its `.f`
- Shrinking `max_t` after a hit is what makes it fast

Checkpoint: rays

```
python main.py --mode rays
```

```
226/1000 rays hit the mesh; brute force and tree agree on 1000/1000 rays.

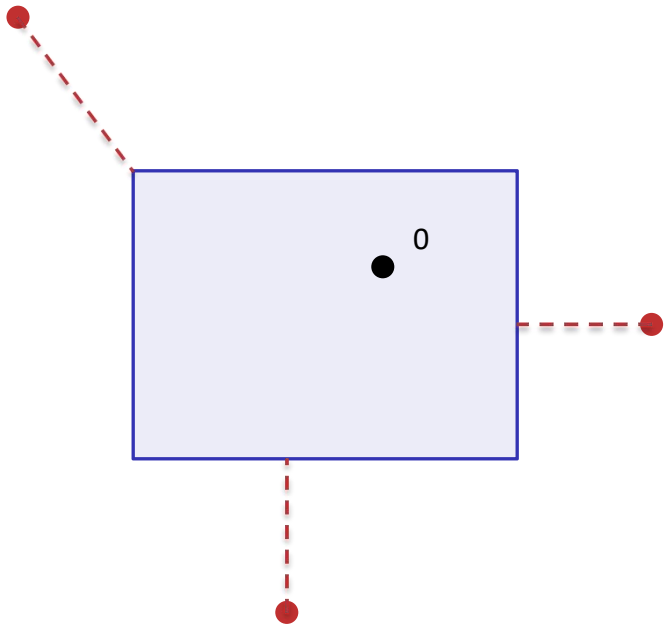
| Method      | Time in seconds |
| brute force |     8.308086700 |
| build tree  |     0.021958800 |
| use tree    |     0.079779900 |
```

- Must **agree** on every ray; you should see a big speed-up
- Disagreement? → re-check `ray_intersect_box` edge cases first

7, 8 · Nearest point: building blocks

`src/nearest_neighbor_brute_force.py` (6) · `src/point_box_squared_distance.py` (6)

- **7**: squared distance to every point, return the smallest
  - Ties → lowest index
- **8**: distance from a point to a **solid** box
  - Per axis: how far outside the slab? (0 if inside)



corner, edge or face — or 0 inside

```
    d_c = ???    # per axis, 0 when inside [min_corner[c], max_corner[c]]
    return sum of d_c ** 2
```

9 · Why not just recurse?

`src/point_AABBTree_squared_distance.py` · 8 pts

test: python run_tests.py --only point_AABBTree_squared_distance

- A ray **misses** most boxes → depth-first prunes a lot
- A point has a distance to **every** box → nothing is "missed"
- DFS dives into the first child even when the answer is elsewhere
- Instead: always expand the **closest** box seen so far → priority queue

9 · Priority-queue search

```
counter = itertools.count()
Q = [(dist(query, root.box), next(counter), root)]
best_d, best = max_sqrd, None
while Q:
    d, _, node = heapq.heappop(Q)
    if d >= best_d: ???                    # prune
    if node is an AABBTree:
        push each non-None child with its box distance
    else:                                  # leaf: CloudPoint
        if d is in [min_sqrd, best_d): update best_d, best
return best is not None, best_d, best
```

- Why `next(counter)`? equal distances → Python compares the nodes → `TypeError`
- Range is `[min_sqrd, max_sqrd)`: < vs <= matters (distance can be 0)
- **Not recursive** — a loop over the heap

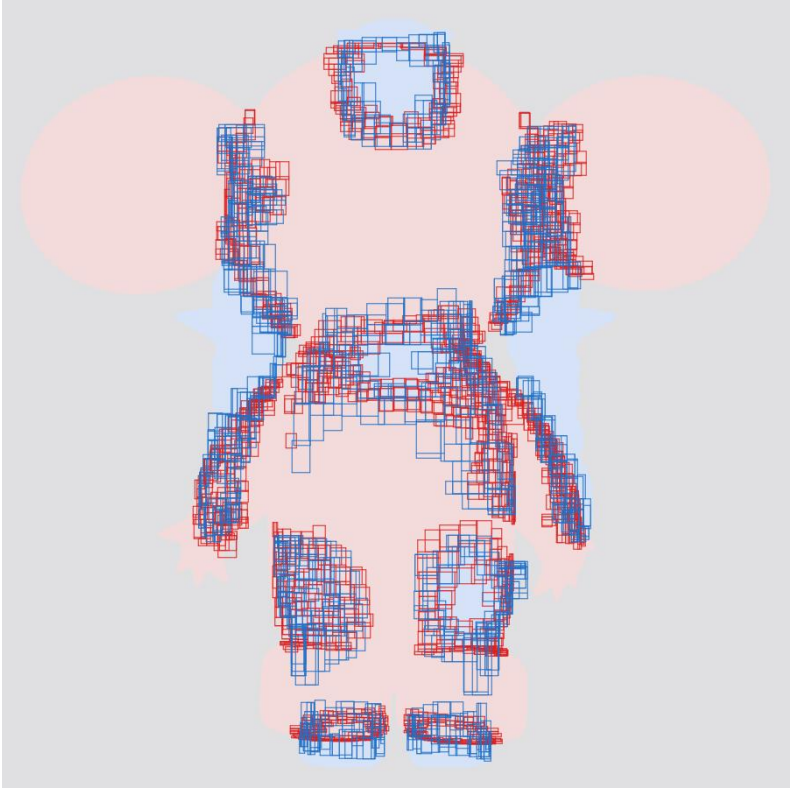
11 · All overlapping pairs between two trees

`src/find_all_intersecting_pairs_using_AABBTrees.py` · 3 pts

test: python run_tests.py --only find_all_intersecting_pairs_using_AABBTrees

- Brute force: every triangle of A vs every triangle of B → O(n·m)
- Walk **both** trees at once
- Only keep a pair of nodes if their boxes overlap
- Result: pairs of **leaf** objects `(leafA, leafB)`

This is the broad phase: candidates only



11 · Pairs: structure

```
Q = deque()
if rootA.box and rootB.box overlap:  Q.append((rootA, rootB))
while Q:
    a, b = Q.popleft()
    if a and b are both leaves:  record (a, b)
    elif a is a leaf:            try (a, each child of b)
    elif b is a leaf:            try (each child of a, b)
    else:                        try (each child of a, each child of b)
    # "try" = append only if both boxes overlap
return pairs
```

- "Leaf" = anything that is **not** an `AABBTree`
- Children can be `None` — the README sketch assumes a full tree
- Check with `python main.py --mode pairs`